

NORAIZ RANA

Task #11

Topic

React Events
and Handling Forms

Date:

21 July 2025

SUBMITTED TO:

APPVERSE TECHNOLOGIES

1. Introduction

In React, handling user interactions like clicks, typing, and form submissions is essential for building interactive web applications. React provides its own **synthetic event system** to handle events efficiently and consistently across all browsers. Additionally, **form handling** in React involves managing inputs through component state and updating the UI in real-time as users interact with the form.

2. React Events

React events are similar to DOM events but are wrapped in the **SyntheticEvent** system for cross-browser compatibility.

Example of an Event Handler:

```
function ClickButton() {  
  function handleClick() {  
    alert("Button was clicked!");  
  }  
  
  return <button onClick={handleClick}>Click  
Me</button>;  
}
```

Key Concepts:

- Events are attached using camelCase (e.g., `onClick`, `onChange`)
- Event handlers are passed as functions, not strings
- React automatically provides the event object as a parameter

Common Events in React:

Event Type	React Event	Description
Mouse	onClick	Fires when element is clicked
Keyboard	onKeyDown	Fires when key is pressed
Form	onSubmit	Fires on form submission
Input	onChange	Fires when input value changes

3. Handling Forms in React

Form handling in React is often done using **controlled components**, where input elements derive their value from React state.

Example of Controlled Input:

```
import { useState } from 'react';

function ContactForm() {
  const [name, setName] = useState("");

  function handleSubmit(event) {
    event.preventDefault();
    alert(`Form submitted by ${name}`);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={name}
```

```

        onChange={ (e) =>
setName(e.target.value) }
        placeholder="Enter your name"
      />
      <button type="submit">Submit</button>
    </form>
  ) ;
}

```

Key Features:

- Input field is **controlled** using `useState`
- `onChange` updates the state on each keystroke
- `onSubmit` prevents default browser reload and handles form logic

4. Uncontrolled vs Controlled Components

Feature	Controlled Component	Uncontrolled Component
Data source	React state	DOM (via <code>ref</code>)
React Control	Full control over input	Minimal React control
Use Case	Dynamic input validation, real-time UI	Simple or third-party libraries